

AI LAB ASSIGNMENT -1 2.1

NAME: B. VISHNU VARDHAN

ROLL NO: 2403A510F2

BATCH NO: 06

Task 1: Sorting – Merge Sort Implementation

Prompt: Generate a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Include a docstring with time and space complexity.

Python Code:

```
from typing import List  Untitled-1  ●
1  from typing import List
2
3  def merge_sort(arr: List[int]) -> List[int]:
4      """
5          Sorts a list in ascending order using Merge Sort.
6
7          Time Complexity:
8              Best Case:  $O(n \log n)$ 
9              Average Case:  $O(n \log n)$ 
10             Worst Case:  $O(n \log n)$ 
11          Space Complexity:  $O(n)$ 
12      """
13      if len(arr) <= 1:
14          return arr
15
16      mid = len(arr) // 2
17      left = merge_sort(arr[:mid])
18      right = merge_sort(arr[mid:])
19
20      merged = []
21      i = j = 0
22
23      while i < len(left) and j < len(right):
24          if left[i] < right[j]:
25              merged.append(left[i])
26              i += 1
27          else:
28              merged.append(right[j])
29              j += 1
30
31      merged.extend(left[i:])
32      merged.extend(right[j:])
33
34      return merged
35
36  # Test
37  arr = [38, 27, 43, 3, 9, 82, 10]
38  print("Sorted Array:", merge_sort(arr))
39
```

AI LAB ASSIGNMENT -1 2.1

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\AI LAB> python -u "d:\AI LAB\tempCodeRunnerFile.python"
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
PS D:\AI LAB>
```

Observation: Merge Sort is efficient and stable, ideal for large datasets. It consistently achieves $O(n \log n)$ complexity, though it uses additional space.

Task 2: Searching – Binary Search with AI Optimization

Prompt: Create a Python function `binary_search(arr, target)` that searches a sorted list and returns the index of the target or -1 if not found. Include a docstring with time complexity details.

Python Code:

AI LAB ASSIGNMENT -1 2.1

```
from typing import List  Untitled-1  from typing import List  Untitled-2
1  from typing import List
2
3  def binary_search(arr: List[int], target: int) -> int:
4      """
5          Searches for a target in a sorted list using Binary Search.
6
7          Parameters:
8          arr (List[int]): Sorted list to search
9          target (int): Element to find
10
11         Returns:
12         int: Index of target or -1 if not found
13
14         Time Complexity:
15             Best Case: O(1)
16             Average Case: O(log n)
17             Worst Case: O(log n)
18         Space Complexity: O(1)
19         """
20         left, right = 0, len(arr) - 1
21         while left <= right:
22             mid = (left + right) // 2
23             if arr[mid] == target:
24                 return mid
25             elif arr[mid] < target:
26                 left = mid + 1
27             else:
28                 right = mid - 1
29         return -1
30
31 # Test
32 sorted_arr = [3, 9, 10, 27, 38, 43, 82]
33 print("Index of 27:", binary_search(sorted_arr, 27))
34 print("Index of 50:", binary_search(sorted_arr, 50))
35
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\AI LAB> python -u "d:\AI LAB\tempCodeRunnerFile.python"
Index of 27: 3
Index of 50: -1
PS D:\AI LAB>
```

Observation: Binary Search is highly efficient for sorted datasets, offering $O(\log n)$ performance. It is suitable for quick lookups in large inventories.

AI LAB ASSIGNMENT -12.1

Task 3: Real-Time Application – Inventory Management System

Prompt: Suggest efficient search and sort algorithms for a retail store inventory system with thousands of products. Implement Python functions for searching by ID or name and sorting by price or quantity.

Algorithm Recommendation Table:

Operation	Recommended Algorithm	Justification
Search by ID	Binary Search (sorted by ID)	Fast $O(\log n)$ search for frequent queries
Search by Name	Hash Table / Dictionary	Constant time $O(1)$ lookup
Sort by Price	Merge Sort	Efficient $O(n \log n)$, stable sorting for large inventory
Sort by Quantity	Merge Sort	Efficient for large datasets, stable

Python Code:

AI LAB ASSIGNMENT -1 2.1

```
from typing import List, Dict
1  from typing import List, Dict
2
3  class Product:
4      def __init__(self, product_id: int, name: str, price: float, quantity: int):
5          self.product_id = product_id
6          self.name = name
7          self.price = price
8          self.quantity = quantity
9
10 # Sample Inventory
11 inventory = [
12     Product(101, 'Shampoo', 5.5, 50),
13     Product(102, 'Soap', 1.5, 100),
14     Product(103, 'Toothpaste', 2.5, 75)
15 ]
16
17 # Search by ID using Binary Search
18 def search_by_id(products: List[Product], target_id: int) -> Product:
19     sorted_products = sorted(products, key=lambda x: x.product_id)
20     left, right = 0, len(sorted_products) - 1
21     while left <= right:
22         mid = (left + right) // 2
23         if sorted_products[mid].product_id == target_id:
24             return sorted_products[mid]
25         elif sorted_products[mid].product_id < target_id:
26             left = mid + 1
27         else:
28             right = mid - 1
29     return None
30
31 # Search by Name using Dictionary
32 def build_name_index(products: List[Product]) -> Dict[str, Product]:
33     return {p.name.lower(): p for p in products}
34
35 # Sort by Price or Quantity using Merge Sort
36 def merge_sort_products(products: List[Product], key_attr: str) -> List[Product]:
37     if len(products) <= 1:
38         return products
39     mid = len(products) // 2
40     left = merge_sort_products(products[:mid], key_attr)
41     right = merge_sort_products(products[mid:], key_attr)
42     merged = []
43     i = j = 0
44     while i < len(left) and j < len(right):
45         if getattr(left[i], key_attr) < getattr(right[j], key_attr):
46             merged.append(left[i])
47             i += 1
48         else:
49             merged.append(right[j])
50             j += 1
51     merged.extend(left[i:])
52     merged.extend(right[j:])
53     return merged
54
55 # Tests
56 print("Search by ID (102):", vars(search_by_id(inventory, 102)))
57 name_index = build_name_index(inventory)
58 print("Search by Name ('soap'):", vars(name_index['soap']))
59 sorted_by_price = merge_sort_products(inventory, 'price')
60 print("Sorted by Price:", [vars(p) for p in sorted_by_price])
61
```

Output:

AI LAB ASSIGNMENT -1 2.1

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\AI LAB> python -u "d:\AI LAB\tempCodeRunnerFile.py"
Search by ID (102): {'product_id': 102, 'name': 'Soap', 'price': 1.5, 'quantity': 100}
Search by Name ('soap'): {'product_id': 102, 'name': 'Soap', 'price': 1.5, 'quantity': 100}
Sorted by Price: [{'product_id': 102, 'name': 'Soap', 'price': 1.5, 'quantity': 100}, {'product_id': 103, 'name': 'Toothpaste', 'price': 2.5, 'quantity': 75}, {'product_id': 101, 'name': 'Shampoo', 'price': 5.5, 'quantity': 50}]
PS D:\AI LAB>
```

Observation: Using Binary Search for ID and a hash table for names ensures fast lookups. Merge Sort handles large inventory efficiently for sorting by price or quantity. This setup balances quick searches and stable sorting for thousands of products.